

Structuri de Date și Algoritmi (SDA)

As.Drd.Ing. Bianca Gușită

AC C/TI, AN II

LAB 2 – SCHEME LOGICE/TDA

Scheme logice si tipuri de date abstracte (TDA)

- Scopul acestei lucrări de laborator este de a prezenta și de a exemplifica:
 - noțiunea de algoritm
 - noțiunea de schema logică
 - noțiunea de tip de date abstract
- Obiectivele lucrării:
 - Reprezentarea sub formă de schemă logică a unor algoritmi simpli
 - Implementarea unor algoritmi diferiți pentru rezolvarea unei probleme date
 - Definirea unor tipuri de date abstracte

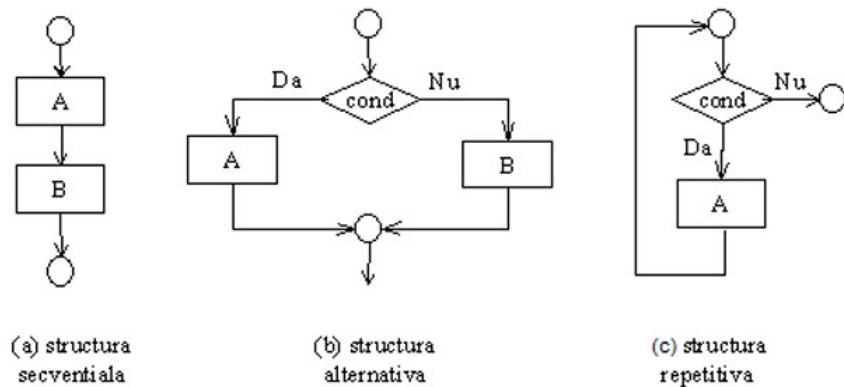
Scheme logice si tipuri de date abstracte (TDA)

- *Algoritmul desemnează o mulțime finită de operații, complet ordonată în timp, care pornind de la date de intrare produce într-un timp finit date de ieșire.*
 - Cu alte cuvinte, algoritmul reprezintă o "rețetă" de rezolvare a unei probleme.
 - Descrierea unui algoritm pentru rezolvarea unei probleme se poate face prin scheme logice sau într-un limbaj de descriere a algoritmilor, numit și pseudocod.

Scheme logice si tipuri de date abstracte (TDA)

- Schema logică este un mijloc de descriere a algoritmilor prin reprezentare grafică.
- Regulile de control ale algoritmului sunt descrise prin blocuri (figuri geometrice) reprezentând operațiile (pașii) algoritmului, iar ordinea lor de aplicare (succesiunea operațiilor) este indicată prin săgeți.
- Fiecărui tip de operație îi este consacrată o figură geometrică (un bloc tip) în interiorul căreia se va înscris operația din pasul respectiv.

Orice algoritm poate fi descris într-o schemă logică folosind următoarele trei structuri de control:

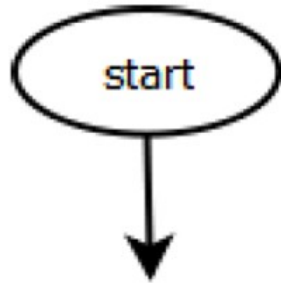


Scheme logice si tipuri de date abstracte (TDA)

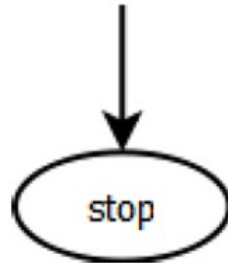
- BLOCURI

Blocurile din schemele logice - se referă la operațiile de bază

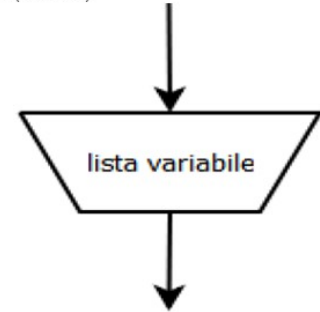
•De start



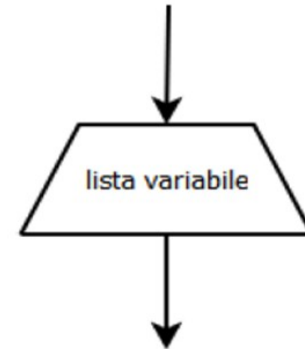
•De stop



•De intrare (de citire)

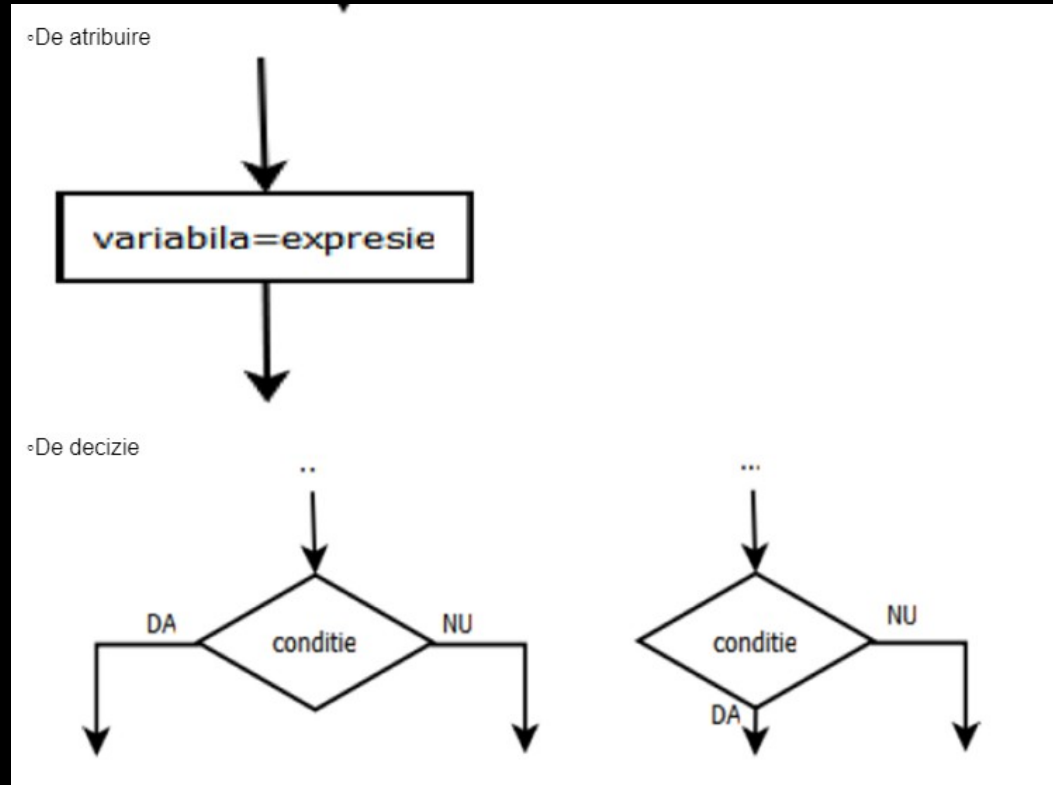


•De ieșire (de scriere)



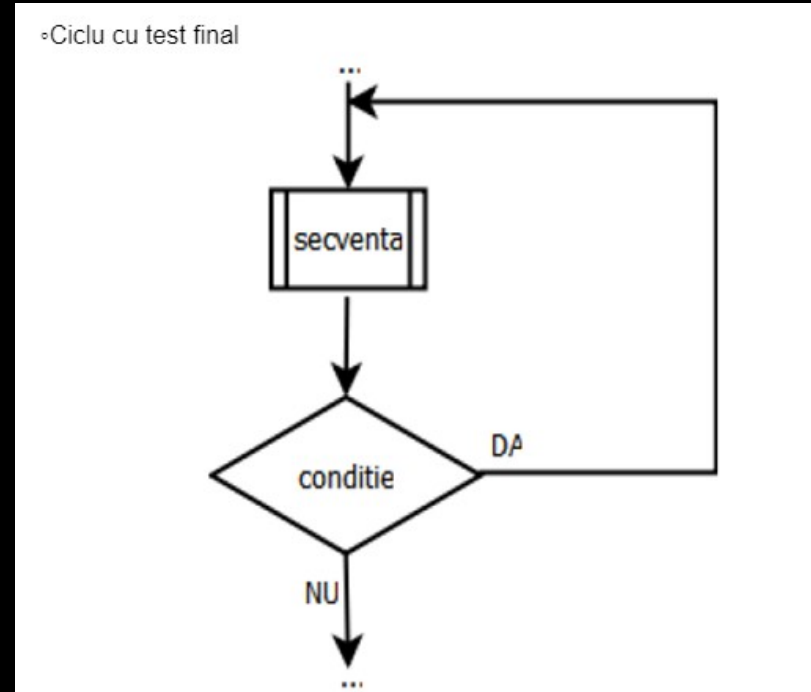
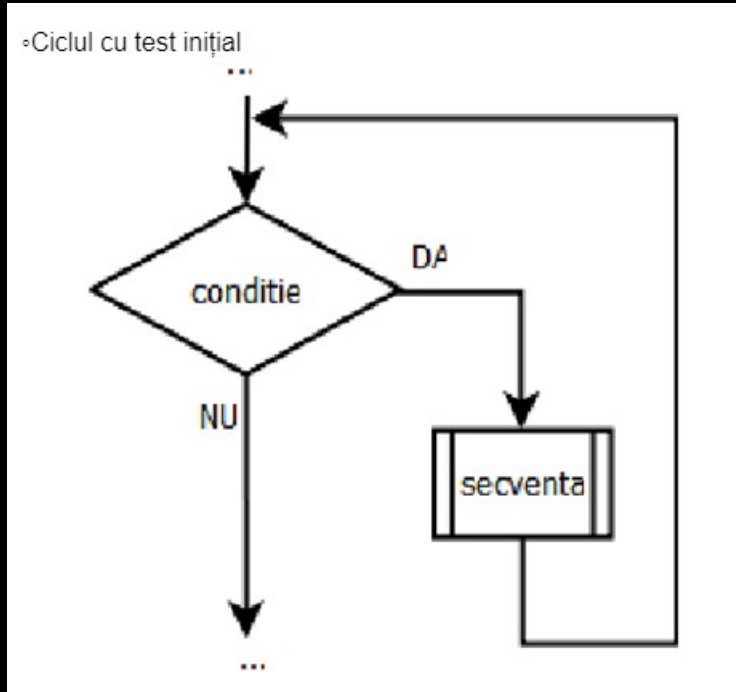
Scheme logice si tipuri de date abstracte (TDA)

- BLOCURI



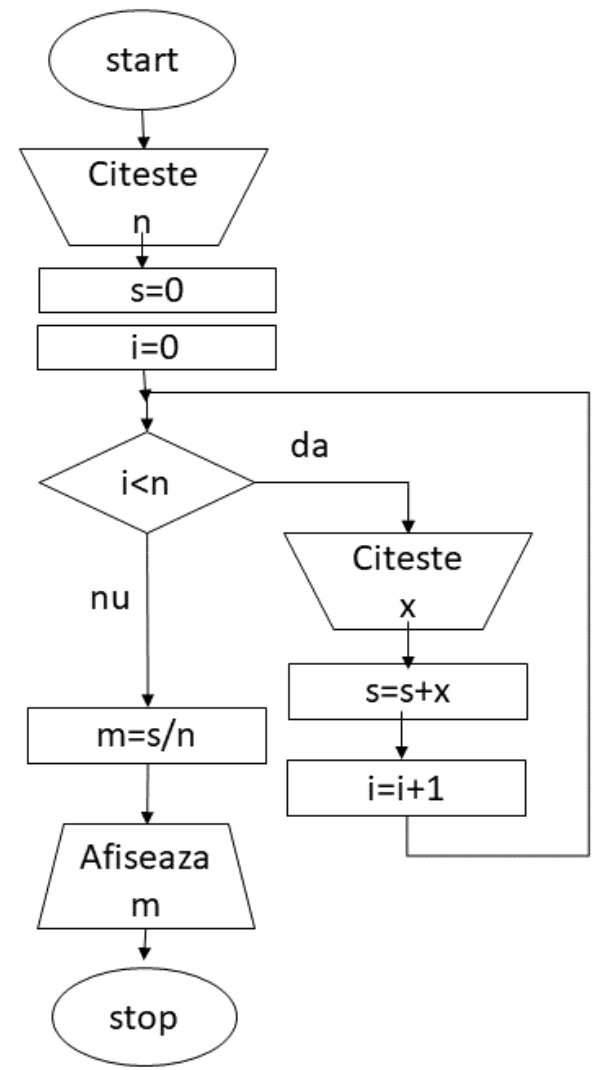
Scheme logice si tipuri de date abstracte (TDA)

- BLOCURI



Scheme logice si tipuri de date abstracte (TDA)

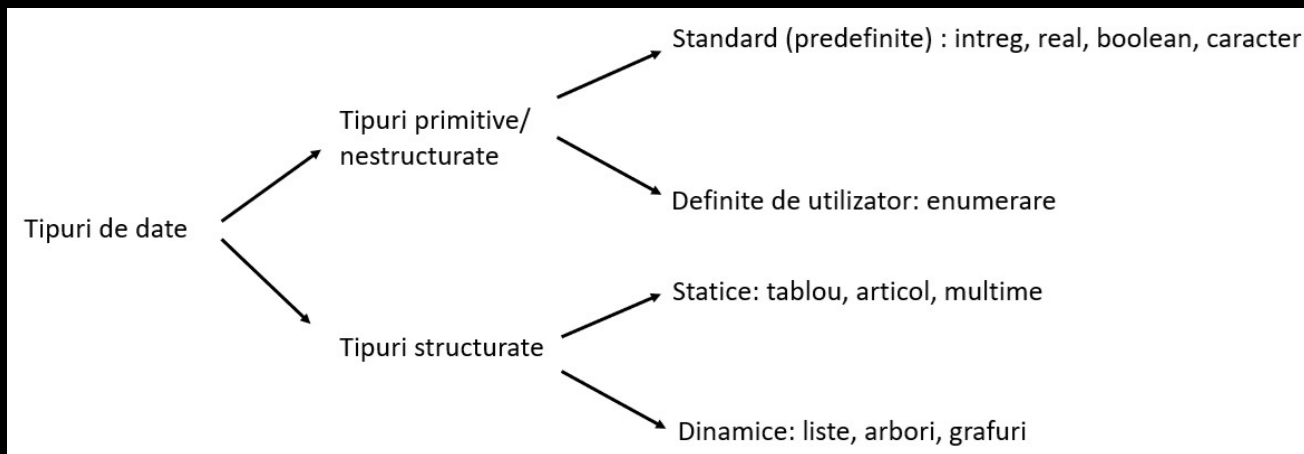
- EXEMPLU
 - Media a n numere.
 - Schema logică



Scheme logice si tipuri de date abstracte (TDA)

Tipuri de date abstracte (TDA)

- Un **tip** de date este o colecție de **valori** la care se asociază o serie de **operații**.



- Definirea unui **tip de date abstract** presupune
 1. Precizarea **modelului matematic**
 2. Definirea **operatorilor** asociați

Scheme logice si tipuri de date abstracte (TDA)

Tipuri de date
abstracte (TDA)

- Implementarea
statică (cu
tablouri de
dimensiune fixă)

```
1  #include <stdio.h>
2  #include <stdbool.h>
3
4  #define MAX 100
5
6  typedef struct {
7      int elements[MAX];
8      int size;
9  } Multime;
10
11 // Verifică dacă un element se află în mulțime
12 bool apartine(Multime *m, int elem) {
13     for (int i = 0; i < m->size; i++) {
14         if (m->elements[i] == elem) {
15             return true;
16         }
17     }
18     return false;
19 }
20
21 // Adaugă un element în mulțime
22 void adauga(Multime *m, int elem) {
23     if (!apartine(m, elem) && m->size < MAX) {
24         m->elements[m->size++] = elem;
25     } else {
26         printf("Elementul %d exista deja sau multimea este plina.\n", elem);
27     }
28 }
```

Scheme logice si tipuri de date abstracte (TDA)

Tipuri de date abstracte (TDA)

- Implementarea
statică (cu tablouri de
dimensiune fixă)

```
30 // Elimină un element din mulțime
31 void elimina(Multime *m, int elem) {
32     for (int i = 0; i < m->size; i++) {
33         if (m->elements[i] == elem) {
34             for (int j = i; j < m->size - 1; j++) {
35                 m->elements[j] = m->elements[j + 1];
36             }
37             m->size--;
38             return;
39         }
40     }
41     printf("Elementul %d nu se afla in multime.\n", elem);
42 }
43 // Afișează elementele din mulțime
44 void afiseaza(Multime *m) {
45     printf("Multimea: ");
46     for (int i = 0; i < m->size; i++) {
47         printf("%d ", m->elements[i]);
48     }
49     printf("\n");
50 }
51
52 int main() {
53     Multime m = {{0}, 0};
54
55     adauga(&m, 5);
56     adauga(&m, 10);
57     adauga(&m, 15);
58     afiseaza(&m);
59
60     elimina(&m, 10);
61     afiseaza(&m);
62
63     return 0;
64 }
```

Scheme logice si tipuri de date abstracte (TDA)

Tipuri de date
abstracte (TDA)

- Implementarea
dinamică (cu
tablouri
dinamice)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <stdbool.h>
4
5 typedef struct {
6     int *elements;
7     int size;
8     int capacity;
9 } Multime;
10
11 // Creează o mulțime goală cu capacitate inițială
12 Multime* creeazaMultime(int capacitate) {
13     Multime *m = (Multime*)malloc(sizeof(Multime));
14     m->elements = (int*)malloc(capacitate * sizeof(int));
15     m->size = 0;
16     m->capacity = capacitate;
17     return m;
18 }
19
20 // Extinde capacitatea mulțimii dacă e nevoie
21 void extinde(Multime *m) {
22     m->capacity *= 2;
23     m->elements = (int*)realloc(m->elements, m->capacity * sizeof(int));
24 }
25
26 // Verifică dacă un element se află în mulțime
27 bool apartine(Multime *m, int elem) {
28     for (int i = 0; i < m->size; i++) {
29         if (m->elements[i] == elem) {
30             return true;
31         }
32     }
33     return false;
34 }
```

Scheme logice si tipuri de date abstracte (TDA)

Tipuri de date abstracte (TDA)

- Implementarea
dinamică (cu tablouri
dinamice)

```
36 // Adaugă un element în mulțime
37 void adauga(Multime *m, int elem) {
38     if (!apartine(m, elem)) {
39         if (m->size == m->capacity) {
40             extinde(m);
41         }
42         m->elements[m->size++] = elem;
43     } else {
44         printf("Elementul %d exista deja in multime.\n", elem);
45     }
46 }
47
48 // Elimină un element din mulțime
49 void elimina(Multime *m, int elem) {
50     for (int i = 0; i < m->size; i++) {
51         if (m->elements[i] == elem) {
52             for (int j = i; j < m->size - 1; j++) {
53                 m->elements[j] = m->elements[j + 1];
54             }
55             m->size--;
56             return;
57         }
58     }
59     printf("Elementul %d nu se afla in multime.\n", elem);
60 }
61
62 // Afișează elementele din mulțime
63 void afiseaza(Multime *m) {
64     printf("Multimea: ");
65     for (int i = 0; i < m->size; i++) {
66         printf("%d ", m->elements[i]);
67     }
68     printf("\n");
69 }
```

Scheme logice si tipuri de date abstracte (TDA)

Tipuri de date abstracte
(TDA)

- Implementarea
dinamică (cu tablouri
dinamice)

```
71 // Eliberează memoria folosită de mulțime
72 void distrugeMultime(Multime *m) {
73     free(m->elements);
74     free(m);
75 }
76
77 int main() {
78     Multime *m = creeazaMultime(2);
79
80     adauga(m, 5);
81     adauga(m, 10);
82     adauga(m, 15);
83     afiseaza(m);
84
85     elimina(m, 10);
86     afiseaza(m);
87
88     distrugeMultime(m);
89     return 0;
90 }
```

Scheme logice si tipuri de date abstracte (TDA)

Tipuri de date
abstracte (TDA)

- Implementare
cu listă simplu
înlănțuită

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <stdbool.h>
4
5  typedef struct Nod {
6      int valoare;
7      struct Nod *urmator;
8  } Nod;
9
10 typedef struct {
11     Nod *primul;
12 } Multime;
13
14 // Creează un nod nou
15 Nod* creeazaNod(int valoare) {
16     Nod *nod = (Nod*)malloc(sizeof(Nod));
17     nod->valoare = valoare;
18     nod->urmator = NULL;
19     return nod;
20 }
21
22 // Creează o mulțime goală
23 Multime* creeazaMultime() {
24     Multime *m = (Multime*)malloc(sizeof(Multime));
25     m->primul = NULL;
26     return m;
27 }
```

Scheme logice si tipuri de date abstracte (TDA)

Tipuri de date abstracte (TDA)

- Implementare
cu listă simplu
înlănțuită

```
29 // Verifică dacă un element se află în mulțime
30 bool apartine(Multime *m, int valoare) {
31     Nod *curent = m->primul;
32     while (curent != NULL) {
33         if (curent->valoare == valoare) {
34             return true;
35         }
36         curent = curent->urmator;
37     }
38     return false;
39 }
40
41 // Adaugă un element în mulțime (doar dacă nu există deja)
42 void adauga(Multime *m, int valoare) {
43     if (!apartine(m, valoare)) {
44         Nod *nou = creeazaNod(valoare);
45         nou->urmator = m->primul;
46         m->primul = nou;
47     } else {
48         printf("Elementul %d exista deja in multime.\n", valoare);
49     }
50 }
```

Scheme logice si tipuri de date abstracte (TDA)

Tipuri de date abstracte (TDA)

- Implementare
cu listă simplu
înlănțuită

```
52 // Elimină un element din multime
53 void elimina(Multime *m, int valoare) {
54     Nod *curent = m->primul;
55     Nod *anterior = NULL;
56
57     while (curent != NULL) {
58         if (curent->valoare == valoare) {
59             if (anterior == NULL) {
60                 m->primul = curent->urmator;
61             } else {
62                 anterior->urmator = curent->urmator;
63             }
64             free(curent);
65             return;
66         }
67         anterior = curent;
68         curent = curent->urmator;
69     }
70     printf("Elementul %d nu se afla in multime.\n", valoare);
71 }
72
73 // Afișează elementele din multime
74 void afiseaza(Multime *m) {
75     Nod *curent = m->primul;
76     printf("Multimea: ");
77     while (curent != NULL) {
78         printf("%d ", curent->valoare);
79         curent = curent->urmator;
80     }
81     printf("\n");
82 }
```

Scheme logice si tipuri de date abstracte (TDA)

Tipuri de date abstracte (TDA)

- Implementare
cu listă simplu
înlănțuită

```
84 // Eliberează memoria folosită de multime
85 void distrugeMultime(Multime *m) {
86     Nod *curent = m->primul;
87     while (curent != NULL) {
88         Nod *temp = curent;
89         curent = curent->urmator;
90         free(temp);
91     }
92     free(m);
93 }
94
95 int main() {
96     Multime *m = creeazaMultime();
97
98     adauga(m, 5);
99     adauga(m, 10);
100    adauga(m, 15);
101    afiseaza(m);
102
103    elimina(m, 10);
104    afiseaza(m);
105
106    distrugeMultime(m);
107    return 0;
108 }
```

Scheme logice si tipuri de date abstracte (TDA)

Tipuri de date
abstracte (TDA)

- Implementare
cu listă dublu
înlănțuită

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <stdbool.h>
4
5 typedef struct Nod {
6     int valoare;
7     struct Nod *urmator;
8     struct Nod *anterior;
9 } Nod;
10
11 typedef struct {
12     Nod *primul;
13     Nod *ultimul;
14 } Multime;
15
16 // Creează un nod nou
17 Nod* creeazaNod(int valoare) {
18     Nod *nod = (Nod*)malloc(sizeof(Nod));
19     nod->valoare = valoare;
20     nod->urmator = NULL;
21     nod->anterior = NULL;
22     return nod;
23 }
24
25 // Creează o multime goală
26 Multime* creeazaMultime() {
27     Multime *m = (Multime*)malloc(sizeof(Multime));
28     m->primul = NULL;
29     m->ultimul = NULL;
30     return m;
31 }
```

Scheme logice si tipuri de date abstracte (TDA)

Tipuri de date abstracte (TDA)

- Implementare
cu listă dublu
înlănțuită

```
33 // Verifică dacă un element se află în mulțime
34 bool apartine(Multime *m, int valoare) {
35     Nod *curent = m->primul;
36     while (curent != NULL) {
37         if (curent->valoare == valoare) {
38             return true;
39         }
40         curent = curent->urmator;
41     }
42     return false;
43 }
44
45 // Adaugă un element în mulțime (doar dacă nu există deja)
46 void adauga(Multime *m, int valoare) {
47     if (!apartine(m, valoare)) {
48         Nod *nou = creeazaNod(valoare);
49         if (m->primul == NULL) {
50             m->primul = nou;
51             m->ultimul = nou;
52         } else {
53             m->ultimul->urmator = nou;
54             nou->anterior = m->ultimul;
55             m->ultimul = nou;
56         }
57     } else {
58         printf("Elementul %d exista deja in multime.\n", valoare);
59     }
60 }
```

Scheme logice si tipuri de date abstracte (TDA)

Tipuri de date abstracte (TDA)

- Implementare
cu listă dublu
înlănțuită

```
62 // Elimină un element din mulțime
63 void elimina(Multime *m, int valoare) {
64     Nod *curent = m->primul;
65
66     while (curent != NULL) {
67         if (curent->valoare == valoare) {
68             if (curent->anterior != NULL) {
69                 curent->anterior->urmator = curent->urmator;
70             } else {
71                 m->primul = curent->urmator;
72             }
73             if (curent->urmator != NULL) {
74                 curent->urmator->anterior = curent->anterior;
75             } else {
76                 m->ultimul = curent->anterior;
77             }
78             free(curent);
79             return;
80         }
81         curent = curent->urmator;
82     }
83     printf("Elementul %d nu se afla in multime.\n", valoare);
84 }
85
86 // Afișează elementele din mulțime
87 void afiseaza(Multime *m) {
88     Nod *curent = m->primul;
89     printf("Multimea: ");
90     while (curent != NULL) {
91         printf("%d ", curent->valoare);
92         curent = curent->urmator;
93     }
94     printf("\n");
95 }
```

Scheme logice si tipuri de date abstracte (TDA)

Tipuri de date abstracte (TDA)

- Implementare
cu listă dublu
înlănțuită

```
97 // Eliberează memoria folosită de multime
98 void distrugeMultime(Multime *m) {
99     Nod *curent = m->primul;
100     while (curent != NULL) {
101         Nod *temp = curent;
102         curent = curent->urmator;
103         free(temp);
104     }
105     free(m);
106 }
107
108 int main() {
109     Multime *m = creeazaMultime();
110
111     adauga(m, 5);
112     adauga(m, 10);
113     adauga(m, 15);
114     afiseaza(m);
115
116     elimina(m, 10);
117     afiseaza(m);
118
119     distrugeMultime(m);
120     return 0;
121 }
```